

AI-Assisted Event Log Generation for Process Mining in Healthcare

Saimir Bala¹[0000–0001–7179–1901], Zeinab Soleimani¹[0009–0006–0103–874X], and
Mathias Weske¹[0000–0002–3346–2442]

Hasso Plattner Institute, University of Potsdam, Potsdam, Germany
{saimir.bala, zeinab.soleimani, mathias.weske}@hpi.de

Abstract. Process mining in healthcare requires structured event logs, yet extracting such logs from complex clinical databases remains a manual, error-prone task requiring substantial SQL skills and domain expertise. This paper investigates the use of large language models to generate SQL queries for event log generation from healthcare data structured according to OMOP, a common data model in healthcare. We develop an artifact to evaluate four prompting strategies across three state-of-the-art LLMs using real-world healthcare data and six clinical treatment processes of varying schema complexity. Generated queries are compared against expert-written ground truth using five metrics: executability, schema conformance, event log completeness, process fidelity, and prompt sensitivity. Results show that prompting strategy has a stronger impact on performance than model choice, while schema complexity is the primary driver of task difficulty. This paper provides a framework as well as practical guidance for LLM-assisted event log generation for process mining from clinical relational data using SQL, further enabling process mining on healthcare data.

Keywords: event log generation · large language models · SQL generation · MIMIC-IV · process mining · healthcare · OMOP CDM

1 Introduction

Process mining enables organisations to discover, monitor, and improve real processes by analyzing event logs extracted from information systems [1]. Healthcare institutions increasingly apply process mining to analyze clinical workflows [2] such as patient pathways, medication administration, and emergency department flow. A prerequisite for any such analysis is an event log that records process instances, activities, and timestamps. In healthcare, this data often originates from electronic health record (EHR) systems and is stored in complex relational databases or clinical data warehouses, where extracting a usable event log requires formulating multi-table SQL queries that demand both clinical domain knowledge and deep familiarity with the underlying schema.

The adoption of standardised clinical data models, most notably the Observational Medical Outcomes Partnership Common Data Model (OMOP CDM) [17],

has partially eased this task by addressing the heterogeneity problem. Over 330 databases in more than 30 countries have been mapped to OMOP CDM [19]. Yet, the event log generation problem persists. In particular, generating reference queries in order to retrieve the data from the correct tables requires substantial manual effort and expertise [6]. To date, the dominant approach in event log generation (ELG) in healthcare remains *rule-based* and *manual*. Large language models (LLMs) have shown strong performance on general text-to-SQL tasks. Recently, LLMs have been applied to the generation of event logs with promising results on well-known databases used in process mining [21]. However, clinical databases introduce unique challenges that concern the resolution of concept vocabulary, indirect foreign-key paths through existing data models (such as the OMOP CDM tables), and domain-specific activity semantics. Whether and how LLMs can navigate these challenges remains an open question.

This paper addresses this gap by designing, implementing and evaluating a framework for LLM-driven SQL generation targeting OMOP CDM-structured clinical databases. The resulting artifact takes as input a natural-language clinical process description and an OMOP CDM schema context, and produces executable SQL that extracts an event log that can be used by process mining algorithms. It comprises a three-phase pipeline: an OMOP-CDM-aware Prompt Builder supporting four prompting strategies of increasing informativeness; an SQL execution layer; and a five-metric evaluation harness covering executability, schema conformance, event log completeness, process fidelity, and prompt sensitivity. We instantiate and evaluate the framework on six clinical treatment processes across three state-of-the-art LLMs and four prompt strategies, using real-world data and expert-authored ground-truth SQL as the reference. The framework is designed to be portable to any OMOP CDM-compliant database.

The remainder of the paper is structured as follows. Section 2 reviews related work. Section 3 formulates the problem statement, and derives research questions and design objectives. Section 4 describes the artifact design, including six clinical treatment processes that instantiate the problem on OMOP CDM. Section 5 presents the experimental evaluation. Section 6 concludes the paper.

2 Related Work

2.1 Event Log Generation in Healthcare

Event log generation (ELG) is a prerequisite for process mining, yet in healthcare it remains a manual, expert-intensive activity. While this task has also been referred to as event log extraction [21], we use the terminology defined by Diba et al. [7], who characterize this endeavor (i.e., ELG) as encompassing three phases: extraction, correlation, and abstraction. They identify healthcare as a domain where all three phases present significant challenges due to data volume, heterogeneity, and clinical complexity. Munoz-Gama et al. [15] confirm in a multi-author roadmap that event log construction is among the most persistent obstacles to adopting process mining in clinical settings.

In practice, this challenge is typically addressed by relying on domain experts to manually specify generation logic, often in the form of hand-written SQL queries. Rebuge and Ferreira [20] present an early methodology for process mining in hospital settings, where event logs are constructed through analyst-defined SQL queries over the hospital information system. Cremerius et al. [6] provide detailed reference queries and configurations for extracting event logs from MIMIC-IV. Their work illustrates that even with a structured clinical database, event log construction requires careful selection and combination of multiple tables and event definitions for each analysis scenario. Park et al. [19] propose a query-based method for constructing event logs from the OMOP CDM, covering inpatient, outpatient, and emergency room processes, and validate the approach using real-world data from a Korean tertiary hospital. More recently, Park et al. [18] extend this line of work to object-centric event logs by proposing a two-phase method for transforming OMOP CDM data into OCEL. Further healthcare ELG contributions include standards-based pipelines for FHIR [10] and the German Medical Informatics Initiative Core Data Set [9], where event logs are constructed through predefined transformation rules over standardized healthcare data representations.

2.2 LLMs as SQL Generators

The task of generating SQL from natural language, text-to-SQL, has emerged as a central benchmark in NLP. The Spider dataset [24], comprising 200 complex multi-table databases, introduced a standard benchmark for cross-domain SQL generation. The more recent BIRD benchmark [14] scales this setting to large, noisy databases with domain-specific values, more closely resembling real-world conditions. Experimental results show that even frontier LLMs still achieve limited execution accuracy, underscoring the difficulty of text-to-SQL on realistic database workloads.

Stein Dani et al. [21] provide the first systematic study of LLM-assisted *event log* extraction, evaluating the generation of SQL queries for process mining across several relational databases, including procurement-to-pay, ERP, and public administration systems. They use gold-standard event logs as references and evaluate the generated logs using precision, recall, and F1-score over event matches. Their setting represents the closest existing baseline to our work. However, the evaluated schemas are conventional relational databases which differ from healthcare data, with the latter adding additional complexity in terms of concept identifiers, use of an additional vocabulary layer and domain semantic.

In the broader process mining context, Vidgof et al. [23] discuss opportunities and challenges of applying LLMs across the BPM lifecycle, while Berti and Qafari [5] experiment with prompting strategies for analyzing process mining artifacts using LLMs. More recently, Berti et al. [4] introduce PM-LLM-Benchmark, evaluating the capabilities of LLMs on a range of process mining reasoning tasks. These works focus on process analysis over existing event logs and models, and do not address the problem of generating SQL queries for ELG from relational data.

2.3 Difficulties Associated With Clinical Data Models

The OMOP CDM [11,17] introduces structural properties absent from the generic schemas used in existing text-to-SQL benchmarks and ELG studies. One key problem is that clinical events are distributed across multiple domain tables, requiring multi-table UNION ALL patterns rather than simple joins. Furthermore, all clinical concepts are encoded as integer `concept_id` values that must be resolved against standardised vocabularies like SNOMED CT, LOINC, etc. It is a challenge to select the correct concept identifiers for the clinical process under study. Moreover, some tables lack a direct foreign key (FK) to the case identifier. For example, `drug_era` aggregates drug exposures at the person level and carries no `visit_occurrence_id`, requiring indirect joins via `person_id` and date-range overlap. These challenges compound: a single clinical treatment process may require all three simultaneously.

Recent work by Gómez-Noé et al. [8] applies LLMs to healthcare process mining, but does not address these challenges. They use LLMs to extract medication mentions from free-text EHR notes, an NLP task distinct from structured SQL generation against a relational schema. Illueca-Fernández et al. [12] present HealthProcessAI, an LLM-enhanced framework for healthcare process analysis that targets process model interpretation and conformance checking, not event log generation. Naeem et al. [16] propose a multi-level ML framework for correlating biomedical activities, but operate on pre-existing event logs rather than generating them from raw clinical data.

In summary, the related work informs us with three findings: *i)* healthcare event log generation remains manually intensive despite data standardization; *ii)* LLMs have demonstrated SQL generation capability on simpler schemas but have not been evaluated on clinical data models; and *iii)* clinical data models such as OMOP CDM introduce additional structural properties that are not captured yet by standard text-to-SQL approaches. Only three papers [16,8,12] tackle all three dimensions of ELG, AI/ML, and healthcare, and none target LLM-driven SQL generation for clinical CDM data. This paper fills this gap.

3 Problem, Research Questions, Design Objectives

Given three challenges identified in the related work, we state the problem of generating an event log for process mining from healthcare data as follows. Given a clinical database $\mathcal{D}$ structured according to the OMOP CDM schema and a natural-language description of a clinical process P , generate an SQL query Q such that executing Q on $\mathcal{D}$ yields a valid event log L , described as a table with three columns: `case_id` (process instance identifier), `activity` (human-readable event name), and `timestamp` (event occurrence time). The generated event log L must be compatible with the XES [22] event log format and faithfully represent the clinical activities defined in P . This task is non-trivial due to the complexity of OMOP CDM, and technical knowledge and domain expertise required.

Given the discussed challenges, the following research questions are derived.

RQ1. Can LLMs generate executable and schema-conformant SQL for healthcare

event log generation from OMOP CDM without manual schema mapping? **RQ2**. How do prompt strategies of increasing informativeness (naive, zero-shot, schema-aware, few-shot) affect SQL executability and event log quality? **RQ3**. How does OMOP CDM schema complexity affect LLM performance across different clinical treatment processes?

To build our artifact, we operationalize the research questions into the following four design objectives. **O1**. Generated SQL must be executable on a standard OMOP CDM instance without manual correction. ($\rightarrow$ *RQ1*) **O2**. Generated SQL queries must produce output with the required three-column structure (`case_id`, `activity`, `timestamp`). ($\rightarrow$ *RQ1*) **O3**. The resulting event log must cover the case population and activity types defined in expert ground-truth SQL, evaluated across clinical treatment processes of varying OMOP CDM schema complexity. ($\rightarrow$ *RQ1*, *RQ3*) **O4**. The framework must quantify result sensitivity across prompt strategies of increasing informativeness, and be replicable with any instruction-following LLM and portable to any OMOP CDM-compliant database. ($\rightarrow$ *RQ2*)

4 Artifact Design

Our artifact is a framework that includes a three-phase pipeline, as illustrated in Figure 1. In the *Prompting* phase, a Prompt Builder is responsible for generating prompts. Its capabilities include the construction of a natural-language request from the OMOP CDM schema and a clinical process description, using four prompting strategies: naive, zero-shot, schema-aware, and few-shot. These prompts are then sent to an LLM via the API. In the *Execution* phase, the SQL query extracted from the LLM API response is run against an OMOP CDM database instance. The result is treated as a candidate event log. In the *Evaluation* phase, the candidate event log and its generating SQL are assessed against five metrics (M1–M5, Section 4.4). The assessment of M3–M5 includes a comparison against expert-written ground-truth SQL.

4.1 Clinical Treatment Processes

To instantiate the problem, we selected six clinical treatment processes that cover a range of OMOP-CDM event-log generation task complexities, clinical domains, and data volumes, summarized in Table 1.

T1 spans hospital-level and unit-level events, requiring a two-granularity union of `visit_occurrence`, `visit_detail`, and `procedure_occurrence` via direct FK paths (*Medium*). **T2** is the hardest structurally: `drug_era` carries no `visit_occurrence_id`, so the join to `drug_exposure` must be performed indirectly via `person_id` and date-range overlap (*High*). **T3** requires a four-table union with disease-specific concept filtering to temporally align diagnosis, lab, medication, and observation events under a single visit (*High*). **T4** presents a heuristic join challenge: `specimen` and `measurement` share no direct

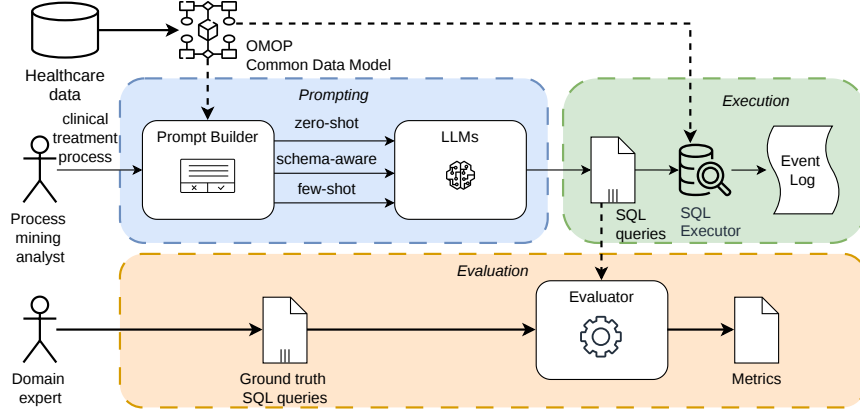


Fig. 1. Approach for LLM-driven SQL generation and evaluation.

FK and must be linked via **person_id** and temporal proximity, risking Cartesian-product explosion (*Medium*). **T5** is the most self-contained: all tables join via **visit_occurrence_id** under a single ED concept filter (*Low*). **T6** was designed as a low-complexity reference: three tables with direct FK paths and a single **concept** vocabulary join to resolve condition names (*Low*). Together, T1–T6 exercise all three structural OMOP CDM challenges (i.e., multi-table composition, indirect joins, and vocabulary resolution).

4.2 Prompt Strategies and LLMs

We evaluate four prompt strategies of increasing informativeness. All prompts (except naive) include the instruction to return *only* the SQL query (no prose, no markdown fences). In all prompts, we ask to produce exactly the three required columns.

Naive. A minimal single-sentence natural-language request with no role framing, no schema hints, no table names, and no concept IDs. We only ask for the three-column (**case_id**, **activity**, **timestamp**) output requirement. Example: “Generate a PostgreSQL query to extract a process mining event log for [process name], with columns case_id, activity, timestamp”. This serves as a lower-bound baseline, testing performance in the absence of any clinical or structural context, consistent with standard NLP benchmarking practices [21,14].

Zero-shot. It provides the clinical process description, the required output schema (**case_id**, **activity**, **timestamp**), and the OMOP CDM table names and relevant concept IDs to use. No column-level schema or foreign-key constraints are supplied. This tests whether process-level guidance alone is sufficient.

Schema-aware. The zero-shot prompt is augmented with the Data Definition Language (DDL) schemas of all relevant OMOP CDM tables: column names, data

Table 1. Clinical treatment processes defined on MIMIC-IV OMOP CDM.

ID	Clinical Treatment Process	Primary OMOP Tables	Complexity
T1	ICU Patient Pathway	visit_occurrence, visit_detail, procedure_occurrence	Medium
T2	Medication Administration	drug_exposure, drug_era	High
T3	Sepsis Treatment Trajectory	condition_occurrence, measurement, drug_exposure, observation	High
T4	Lab-Order-to-Result Cycle	measurement, specimen	Medium
T5	Emergency Department Flow	visit_occurrence, visit_detail, condition_occurrence, procedure_occurrence	Low
T6	Inpatient Diagnosis Pathway	visit_occurrence, condition_occurrence, concept	Low

types, primary keys, and foreign-key relationships. This tests whether explicit structural knowledge improves SQL correctness.

Few-shot. The schema-aware prompt is further augmented with one worked example from a different clinical treatment process (T6 for T1–T5, and T1 for T6), including the full SQL query and its expected output structure. This tests whether in-context exemplars transfer across processes.

We evaluate three LLMs representing distinct capability tiers and access models. GPT-4o (OpenAI): proprietary frontier model, API access. Claude Sonnet 4 (Anthropic): proprietary frontier model, API access. Llama 3.3 70B (Meta): open-weight model, accessed via Groq API. All models are queried at temperature 0 for reproducibility, with a maximum of 4096 output tokens. Each (clinical treatment process, LLM, strategy) combination is repeated three times. This is sufficient to detect non-determinism at temperature 0 and consistent with prior LLM-based ELG evaluation practice [21]. In total, the combination yields 216 SQL queries (6 treatment processes $\times$ 3 LLMs $\times$ 4 strategies $\times$ 3 repetitions).

4.3 Execution Environment

Each LLM response passes through an SQL extractor that strips prose and markdown fences, saving the extracted query as a standalone `.sql` file. All queries are then executed against the MIMIC-IV OMOP CDM on a PostgreSQL instance (`search_path = cdm`) with a per-query timeout to prevent runaway scans. When a query fails, the SQL Executor classifies the error into six categories: *syntax errors*, *missing-table references*, *missing-column references*, *type mismatches*,

timeouts, and *ambiguous column references*. This classification is used in the evaluation (Section 5.2) to distinguish structural from superficial LLM failures.

Our artifact, including all 216 generated SQL files, ground-truth queries, prompt templates, and annotated execution logs are available in the replication package [3]. The repository contains representative SQL outputs (including complete generated queries for each treatment process, LLM, and strategy, as well as illustrative success and schema-hallucination failure cases) and further steps to convert them to XES event logs, along with initial process mining results. The source code is maintained on GitHub <https://github.com/s41m1r/elg-with-ai-experiment>.

4.4 Evaluation Metrics

We define five metrics organised into two groups: *i) Execution metrics* (M1–M2, ground-truth independent) that assess the technical correctness of the generated SQL and its conformance to the required output schema, independent of any ground-truth reference; and *ii) Log quality metrics* (M3–M5, ground-truth dependent) that compare the generated event log against a reference log produced by expert-authored ground-truth SQL, assessing the completeness and fidelity of the extracted process data, as well as the sensitivity of results to prompt design.

Let Q denote a generated SQL query, $L = Q(\mathcal{D})$ the event log it produces when executed on the OMOP CDM database $\mathcal{D}$, and $L^* = Q^*(\mathcal{D})$ the reference log from the expert-authored ground-truth query Q^* . Each log row is a triple (*case_id*, *activity*, *timestamp*). We write $C(\cdot)$ for the set of distinct case identifiers in a log, $A(\cdot)$ for the set of distinct activity labels, and $\text{cols}(\cdot)$ for the set of column names in a result set. We define the metrics as follows.

M1 (Executability). $M1(Q) = 1$ if Q executes successfully on $\mathcal{D}$, and 0 otherwise. Failed queries are further classified by error category as described in Section 4.3.

M2 (Schema Conformance). $M2(Q) = 1$ if $\text{cols}(L) = \{\text{case_id}, \text{activity}, \text{timestamp}\}$; otherwise 0. This checks whether the query produces the required three-column structure. M2 is assessed only for queries with $M1 = 1$.

M3 (Event Log Completeness) consists of three sub-metrics to capture different facets of coverage. **M3a** (case coverage): the fraction of ground-truth cases recovered, $M3a = |C(L) \cap C(L^*)| / |C(L^*)|$. **M3b** (activity type coverage): the fraction of ground-truth activity labels present, $M3b = |A(L) \cap A(L^*)| / |A(L^*)|$. For example, if the ground truth produces activities $\{A, B, C\}$ and the LLM produces $\{A, D, E\}$, then $M3b = 1/3$ even though both logs contain three activity types. **M3c** (volume ratio): the row count ratio, $M3c = |L| / |L^*|$, indicating whether the generated log over- or under-produces events.

M4 (Process Fidelity). Activity Jaccard similarity between the activity type sets: $M4 = |A(L) \cap A(L^*)| / |A(L) \cup A(L^*)|$. In the same example, $M4 = 1/5$ since $|\{A\}| / |\{A, B, C, D, E\}| = 1/5$; $M4$ is strictly $\leq M3b$ because the union in the denominator penalises spurious labels.

M5 (Prompt Sensitivity). For each (LLM, treatment process) pair, we first average each metric across the three repetitions per strategy to obtain $\bar{m}_{k,s}$ (the

mean metric value for strategy s), then compute the standard deviation across the four strategies: $M5_k = \sigma_{s \in \{S_0, S_1, S_2, S_3\}}(\bar{m}_{k,s})$, $k \in \{M1, M3a, M3b, M4\}$. M5 is computed over these four metrics because they are all bounded on a common $[0, 1]$ scale. For M1, the strategy-level mean corresponds to the executability rate, so its standard deviation captures how sensitive executability is to prompt strategy. For M3a, M3b, and M4, the standard deviations capture sensitivity in case coverage, activity-type coverage, and process fidelity, respectively. Their aggregate mean ($\bar{M5}$) is therefore interpreted as a descriptive summary of prompt sensitivity across bounded evaluation dimensions. M2 is excluded because it is only defined for M1-passing queries, so its sample size varies across strategies and its σ would conflate prompt sensitivity with executability filtering. M3c is excluded because it is unbounded (values $\gg 1$ are common), so a single outlier would dominate the composite and mask meaningful variation in the bounded metrics. A composite $\bar{M5}$ is the mean of the four individual sensitivities. Lower values indicate robustness to prompt design. For instance, if an LLM achieves M3a scores of 0.0, 0.83, 1.0, 0.89 across the four strategies for a given task, then $M5_{M3a} = \sigma(\{0.0, 0.83, 1.0, 0.89\}) \approx 0.46$, reflecting high sensitivity driven by the naive strategy’s poor performance.

The introduced metrics are justified as follows: M1 and M2 follow standard text-to-SQL evaluation practice [24, 14]; M3a–M3b adapt the case- and activity-level recall decomposition introduced by Stein Dani et al. [21]; M4 applies Jaccard set similarity, standard in process mining [19]; M5 is a prompt sensitivity measure based on cross-strategy standard deviation, introduced in this work.

5 Evaluation

Next, we present the results and a discussion of the research questions.

5.1 Experimental Setup

Our database setting is the following. We use MIMIC-IV [13] mapped to OMOP CDM, hosted on a PostgreSQL instance (schema `cdm`). The full database contains approximately 190,000 patients. Expert ground-truth SQL was validated on the full instance; ground-truth event-log sizes are 1.84M (T1), 30.9M (T2), 44.9M (T3), 215M (T4), 4.30M (T5), and 2.20M (T6) rows. For M3/M4/M5 evaluation, all LLM-generated and ground-truth queries are executed on a stratified 1,038-patient sample (12 OMOP CDM tables) drawn from the full database, covering all six treatment process cohorts with partial overlap: 400 ICU patients (T1), 350 sepsis patients (T3), 250 ED patients (T5), and 150 inpatient-only patients (T6), yielding 1,038 unique patients after deduplication; T2 and T4 are covered by the ICU/sepsis cohorts. This sample covers 1,038 of the $\sim 190,000$ patients in the full instance ($\approx 0.55\%$), drawn by stratified cohort sampling to guarantee representation of all six treatment processes. M1 executability is assessed on the full database. The reference (ground-truth) queries for all six treatment

processes were validated by one author with combined SQL and clinical-data-modelling expertise with about two years of experience working with EHR data and knowledge of the MIMIC-IV OMOP CDM instance.

Our execution environment is set up as follows. M1 queries are executed from our Executor module, a Python script that uses `psycopg2` against PostgreSQL with a 60-second timeout per query; no manual correction is applied. M3–M5 are computed using DuckDB in-memory on the 1,038-patient CSV sample, using direct aggregate subqueries (`SELECT DISTINCT`) to avoid materialising Cartesian-product result sets. All queries are extracted from LLM responses by stripping prose and markdown fences.

Our experimental grid is the following. The full grid comprises $6 \text{ processes} \times 3 \text{ LLMs} \times 4 \text{ strategies} \times 3 \text{ repetitions} = 216$ query executions. The six treatment processes cover all three structural OMOP CDM challenges at two complexity levels; the three LLMs represent distinct capability tiers (open-weight vs. proprietary) and API access paradigms; the four strategies span the full spectrum from no context (naive) to full schema and in-context exemplar (few-shot). For reproducibility, the exact API model identifiers are `gpt-4o-2024-08-06` (GPT-4o), `claude-sonnet-4-20250514` (Claude Sonnet 4), and `llama-3.3-70b-versatile` (Llama 3.3 70B, served via Groq).

5.2 Results: Executability and Schema Conformance (M1, M2)

Overall, of the 216 generated queries, 179 execute without error on the full MIMIC-IV OMOP CDM PostgreSQL instance, yielding an overall M1 rate of 83%. Among all 179 executable queries, every result set contains the required `case_id`, `activity`, and `timestamp` columns (**M2 = 1.0 for all executed queries**). The collapse of M1 and M2 is a direct consequence of the prompt instruction mandating the three-column output schema. LLMs that generate executable SQL invariably also satisfy the structural constraint.

Next, we discuss the results by looking at the combination of LLM and prompt strategy. Table 2 reports M1 pass rates for all twelve ($3 \text{ LLMs} \times 4 \text{ strategies}$) combinations. Claude Sonnet achieves the highest overall executability (96%), followed by GPT-4o (92%) and Llama 3.3 70B (61%). The few-shot strategy yields the highest overall rate (93%), with schema-aware at 91% and zero-shot at 85%. The naive strategy, as expected, performs worst at 63%: with only generic process-level guidance, LLMs generate queries that target incorrect tables or produce non-event-log structures. Notably, GPT-4o performs well under naive prompting (89%), while Llama 3.3 collapses to 17%, confirming that minimal prompting requires strong prior knowledge of OMOP CDM. Another notable observation is the Llama 3.3 schema-aware performance degradation (72% schema-aware vs. 78% zero-shot).

We also analyze executability by treatment process. Table 3 shows M1 executability pass rates per treatment process ELG task. T5 (ED Flow) is the easiest at 94%, with T1 (ICU) and T3 (Sepsis) close behind at 92%. T6 (Inpatient Diagnosis Pathway), the new low-complexity task, achieves 89% overall, validating its design intent. T2 (Medication Administration) remains the hardest at 64%, driven

Table 2. M1 Executability (%): LLM and prompt strategy. Cell: pass/total (18 runs).

LLM	Naive	Zero-shot	Schema-aware	Few-shot	Overall
GPT-4o	89% (16/18)	78% (14/18)	100% (18/18)	100% (18/18)	92% (66/72)
Claude	83% (15/18)	100% (18/18)	100% (18/18)	100% (18/18)	96% (69/72)
Llama	17% (3/18)	78% (14/18)	72% (13/18)	78% (14/18)	61% (44/72)
Overall	63% (34/54)	85% (46/54)	91% (49/54)	93% (50/54)	83% (179/216)

Table 3. M1 Executability (%) by clinical treatment process (36 runs each).

Task	Process	M1 Pass Rate
T1	ICU Pathway	33/36 92%
T2	Medication Admin.	23/36 64%
T3	Sepsis Trajectory	33/36 92%
T4	Lab-Order Cycle	24/36 67%
T5	ED Flow	34/36 94%
T6	Diagnosis Pathway	32/36 89%
Total		179/216 83%

by consistent LLM hallucination of a non-existent `visit_occurrence_id` foreign key in the `drug_era` table. T4 (Lab-Order Cycle) at 67% reflects continued difficulty with the heuristic specimen-to-measurement join.

The *error categories* of the 37 ($= 216 - 179$) failed queries fall into two main groups: (i) *schema hallucination* and (ii) *syntax errors and other failures*. Schema hallucination affected 20 queries. LLMs referenced columns absent from OMOP CDM, most commonly `drug_era.visit_occurrence_id` (T2, all models) and `specimen.visit_occurrence_id` (T4, Llama 3.3). Syntax errors and other failures affected 17 queries. They came largely from naive-strategy Llama 3.3 queries that generated malformed SQL without adequate structural guidance. Notably, no schema hallucinations involve native MIMIC-IV table names: all models correctly use OMOP CDM table names throughout.

5.3 Results: Log Completeness and Process Fidelity (M3, M4)

Case coverage (M3a). Table 4 reports M3 and M4 results averaged across all six clinical treatment process tasks. Mean case coverage is 66.4% (see overall mean row in Table 4), indicating that LLM-generated queries recover the majority of patient populations defined in the ground-truth SQL. The naive strategy performs poorly on M3a (near 0% for Claude Sonnet and Llama 3.3), confirming that without clinical context the LLMs produce structurally valid but semantically incorrect queries that return wrong patient populations. The zero-shot and schema-aware strategies achieve 100% case coverage for Claude Sonnet, and few-shot achieves it for both GPT-4o and Claude Sonnet. Excluding the naive

Table 4. M3 Completeness and M4 Fidelity results averaged across all six tasks.

LLM	Strategy	M3a (case)	M3b (act.)	M3c (vol.)	M4 (Jacc.)
GPT-4o	Naive	9.5%	0.0%	0.00	0.000
	Zero-shot	83.3%	0.1%	1.48	0.001
	Schema-aware	100.0%	0.1%	5.68	0.001
	Few-shot	88.9%	16.7%	5.23	0.089
Claude Sonnet	Naive	0.0%	0.0%	0.26	0.000
	Zero-shot	100.0%	16.8%	1.43	0.168
	Schema-aware	93.2%	16.8%	0.85	0.168
	Few-shot	100.0%	17.2%	1.18	0.170
Llama 3.3 70B	Naive	0.0%	0.0%	0.00	0.000
	Zero-shot	77.8%	0.1%	2.12	0.001
	Schema-aware	66.7%	0.1%	1.27	0.001
	Few-shot	77.8%	16.8%	1.43	0.090
Overall mean		66.4%	7.1%	1.74	0.057

strategy, mean M3a rises to **87.5%** (versus 66.4% overall), demonstrating that the naive lower bound is the primary drag on the aggregate figure.

Activity label coverage (M3b), key finding. Despite adequate case coverage for structured prompts, *exact* activity label coverage remains near-zero for T1–T5 across most of the combinations (mean M3b = 7.1% averaged over all six tasks, but effectively 0% for T1–T5). The exception is T6, where few-shot GPT-4o and all Claude Sonnet strategies achieve M3b = 1.0, and Llama 3.3 few-shot achieves M3b = 1.0 as well. The T6 result is explained by the fact that T6 explicitly requires a **concept** table join to resolve condition names, a natural-language pattern LLMs reproduce accurately when given structural guidance. For T1–T5, the mismatch arises from *activity label naming conventions*: ground-truth SQL uses OMOP **concept_name** lookups (e.g., 'Inpatient Visit', 'Blood pressure'), while LLMs construct string literals (e.g., 'Hospital Admission', 'ICU Transfer'). These are semantically equivalent but lexically disjoint, yielding zero intersection under exact-match comparison. This confirms the label mismatch problem identified by [21], and extends it to the OMOP CDM setting where the underlying cause is controlled vocabulary dependency rather than schema naming conventions. The T6 result provides mechanistic evidence for this distinction.

Volume ratio (M3c). LLM-generated queries tend to over-produce events relative to the ground truth. Claude Sonnet stays closest to the reference volume (M3c $\approx$ 0.85–1.43 for structured strategies), while GPT-4o schema-aware and few-shot over-produce by a factor of 5–6 $\times$, largely driven by under-constrained joins on high-volume tasks (T2, T4) that produce Cartesian-product row inflation.

Process fidelity (M4). M4 (Jaccard similarity) tracks M3b closely but is strictly lower, because the union denominator penalises spurious activity labels generated by LLMs that do not appear in the ground truth. The overall mean

M4 is 0.057, compared to M3b = 7.1%. The gap between the two metrics is most visible for few-shot strategies, where LLMs achieve non-trivial M3b ($\approx 17\%$) but M4 drops to 0.09–0.17 because the LLM-generated queries introduce additional activity labels absent from the ground truth. As with M3b, T6 is the exception: M4 = 1.0 for the same configurations that achieve perfect M3b, confirming that when activity labels match exactly, no spurious labels are introduced.

Normalisation experiment (M3b, M4). To quantify whether the near-zero M3b and M4 reflect a naming mismatch rather than a structural failure, we conduct a category-level normalisation on T5 (ED Flow). The ground-truth SQL for T5 produces 1,888 distinct activity labels across five categories by concatenating OMOP `concept_name` values (e.g., 'ED Diagnosis: Chest pain', 'ED Procedure: CT scan'). LLM-generated queries produce exactly four category-level labels ('ED Arrival', 'ED Discharge', 'ED Diagnosis Recorded', 'ED Procedure'). We define a manual mapping of five rules: 'ED Arrival' $\rightarrow$ 'ED Arrival' (exact), 'ED Discharge' $\rightarrow$ 'ED Departure' (synonym), 'ED Diagnosis Recorded' $\rightarrow$ 'ED Diagnosis: *' (prefix), 'ED Procedure' $\rightarrow$ 'ED Procedure: *' (prefix), and no mapping for the GT-only category 'ED Sub-Stay: *' (absent from LLM output). Under this category-level comparison, all 27 non-naive T5 queries yield identical exact M3b = 0.053% and normalised M3b = **80%**, a uniform $1,509\times$ improvement (verifiable as $0.80/0.00053 \approx 1,509$). The remaining 20% gap is attributable to a single missing event category (`visit_detail` sub-stays), not to incorrect SQL. This demonstrates that the near-zero exact-match scores are an artefact of the evaluation granularity, not a fundamental limitation: category-level normalisation, achievable through a lightweight post-processing mapping, recovers the vast majority of semantic fidelity.

5.4 Results: Prompt Sensitivity (M5)

Table 5 reports M5 prompt sensitivity as the mean σ across M1, M3a, M3b, M4, broken down by task and LLM. With the inclusion of the naive strategy, sensitivity scores increase substantially compared to a three-strategy evaluation, as the naive lower bound widens the spread. Across LLMs, Llama 3.3 is the most sensitive overall ($\overline{M5} = 0.218$), driven by extreme variability between near-zero naive performance and reasonable structured-strategy results. GPT-4o is moderately sensitive ($\overline{M5} = 0.211$), while Claude Sonnet is the most consistent ($\overline{M5} = 0.188$), reflecting stronger robustness to prompt design. Across tasks, T6 (Inpatient Diagnosis Pathway) is the highest-sensitivity task for every model ($\overline{M5} = 0.382$): few-shot achieves M3b = 1.0 while naive achieves 0.0, creating a near-maximal spread. Conversely, T5 (ED Flow) is the most stable task, with identical sensitivity across all three LLMs ($\overline{M5} = 0.125$), indicating that the prompt strategy has a uniform and limited effect on performance for this task regardless of model choice.

Table 5. M5 Prompt Sensitivity per task and LLM: mean σ across M1, M3a, M3b, M4 (higher = more sensitive to prompt strategy choice). Bold = highest sensitivity per column.

Treatment Process	GPT-4o	Claude Sonnet	Llama 3.3 70B	Mean
T1 ICU Pathway	0.210	0.252	0.238	0.233
T2 Medication Admin.	0.268	0.125	0.096	0.163
T3 Sepsis Trajectory	0.167	0.118	0.250	0.178
T4 Lab-Order-to-Result	0.167	0.130	0.160	0.152
T5 ED Flow	0.125	0.125	0.125	0.125
T6 Inpatient Diagnosis	0.331	0.375	0.441	0.382
Mean	0.211	0.188	0.218	0.206

5.5 Discussion of Results vs. Research Questions

Next, we discuss to what extent our results address each of the research questions.

RQ1: Can LLMs generate executable SQL for healthcare ELG from OMOP CDM? Yes, with important caveats. Frontier proprietary models achieve up to 96% executability (Claude Sonnet) across all four prompt strategies; GPT-4o reaches 92% overall. The primary failure mode is schema hallucination (Section 5.2), consistent and model-agnostic in zero-shot settings. Open-weight models (Llama 3.3 70B, 61%) lag considerably and are less suitable for direct deployment without fine-tuning, particularly with naive prompting (17%).

Regarding event log quality: LLMs reliably identify the correct patient populations for structured prompts (M3a = 66.4% mean case coverage). Notably, T6 demonstrates that low-complexity processes with explicit vocabulary joins are fully solvable, while the near-zero M3b/M4 for T1–T5 reflects activity label naming conventions rather than a fundamental limitation.

The near-zero M3b/M4 for T1–T5 is not addressed by the four prompt strategies evaluated. Schema-aware prompting conveys table structure but not vocabulary content: the OMOP `concept` table contains over 40 million concepts, and no static prompt can enumerate the task-relevant subset. A *vocabulary-aware* strategy, dynamically retrieving and injecting the relevant `concept_name` values, was deliberately excluded for two reasons. First, the four strategies evaluated represent the current standard in text-to-SQL benchmarking [21,14] and the primary goal was to establish a reproducible baseline. Second, healthcare vocabularies are high-volume databases requiring a dedicated experimental setting with higher memory and compute. Vocabulary-aware prompting is the most direct intervention and will be addressed in future work.

RQ2: How do prompt strategies affect SQL quality? The naive strategy performs significantly worse than all structured strategies (63% M1 overall vs. 85–93%), confirming that clinical context is essential. Among structured strategies, few-shot achieves the highest rate (93%), with schema-aware close behind (91%). The few-shot strategy is the most reliable for frontier models; unexpectedly, Llama 3.3 degrades under schema injection (72% schema-aware vs. 78% zero-

shot), suggesting that long DDL schemas exceed the effective instruction-following capacity of open-weight models. M5 sensitivity analysis (Table 5) confirms that the naive lower bound substantially increases variance; Claude Sonnet is the most stable model and Llama 3.3 the most sensitive. Schema-aware prompting is the recommended baseline; few-shot adds value for frontier models on complex tasks.

RQ3: How does schema complexity affect performance? Complexity remains the primary driver of task difficulty. T5, T1, and T3 (low to high complexity but with direct FK paths) achieve the highest executability (92–94%), while T6 (low complexity) follows closely at 89%. T2 and T4 are the hardest: T2 due to the indirect `drug_era` join, T4 due to the heuristic specimen-to-measurement linkage. The T6 result provides a useful contrast: low-complexity processes with explicit concept joins are fully solvable by current frontier LLMs. These structural OMOP CDM properties (absent from Spider [24] and BIRD [14] benchmarks) are a distinct generalisation challenge for text-to-SQL models.

These findings have several implications. Our results show that LLMs perform well on SQL code generation. Non technical users, such as doctors, may now have access to event logs that can readily be used by process mining tools. Process mining practitioners may use our framework to explore datasets from different domains (such as healthcare). Research on event log generation, may use our framework as a systematic benchmark of LLM driven ELG from OMOP data.

Threats to validity. *Internal:* M3/M4/M5 are computed on a stratified 1,038-patient sample; while larger than typical prototyping datasets, this may not capture the full distribution of clinical cases. The ground-truth SQL is authored by a single expert, so the reference logs encode one set of modelling choices on which experts could reasonably differ, and which propagate into M3–M5. For example, the T3 sepsis cohort uses a hand-curated 129-concept list; a different expert curating a narrower or broader concept set would change the reference case population $C(L^*)$ and thus the M3–M5 scores measured against it. Multi-expert ground truth, with agreement measured on the resulting cohort and activity definitions, would strengthen these claims. Activity label comparison uses exact string matching; fuzzy matching or embedding-based similarity would yield higher M3b/M4 and would be more informative. *External:* Three LLMs are evaluated at a single point in time; model updates may alter results. The six clinical treatment processes cover the most common OMOP domain tables but not `device_exposure`, `note`, or `episode`. *Construct:* M3b/M4 near-zero for T1–T5 may underestimate true semantic fidelity: LLM activity labels are clinically meaningful, just differently named. Normalisation could substantially improve these metrics.

Other limitations. Beyond the threats discussed above, the proposed framework has a few practical limitations. The framework requires hospital data to be pre-mapped to OMOP CDM, which remains a significant ETL barrier for the majority of clinical institutions worldwide; results may further vary across OMOP instances where concept vocabulary mapping quality differs. General-purpose LLMs were evaluated throughout. Whether medically fine-tuned models yield better concept resolution on OMOP schemas remains an open question.

6 Conclusion

This paper presents a systematic benchmark of LLM-driven SQL generation for clinical ELG on the OMOP CDM. Across six clinical treatment processes, three LLMs, and four prompting strategies, frontier proprietary models achieve high SQL executability and medium-to-high case coverage. Low-complexity processes with direct vocabulary joins enable near-perfect event log generation, while complex tasks expose an activity labeling gap addressable through category-level normalization. OMOP CDM schema complexity is the primary determinant of task difficulty, largely independent of model choice; naive prompting confirms the importance of contextual information.

These findings provide practical guidance for practitioners. Schema information is essential at prompting time; zero-shot is reliable only for low-complexity processes with clear foreign key paths; frontier models perform best with few-shot prompting. Our framework is portable across OMOP CDM instances and the three-phase ELG approach generalizes beyond healthcare. Future work will address activity label normalization via OMOP vocabulary alignment, and object-centric log generation for processes such as concurrent ICU and ED stays.

Acknowledgments. We thank Lars Rückert, David Weese, and Pablo Guerrero from D4L data4life gGmbH for providing access to the MIMIC-IV OMOP CDM data.

References

1. van der Aalst, W.M.P.: Process Mining - Data Science in Action, Second Edition. Springer (2016)
2. Aversano, L., Iammarino, M., Madau, A., Pirlo, G., Semeraro, G.: Process mining applications in healthcare: a systematic literature review. *PeerJ Comput. Sci.* **11**, e2613 (2025)
3. Bala, S., Soleimani, Z., Weske, M.: AI-Assisted Event Log Extraction for Process Mining in Healthcare (Replication Package) (May 2026). <https://doi.org/10.5281/zenodo.20425398>
4. Berti, A., Kourani, H., van der Aalst, W.M.P.: PM-LLM-Benchmark: Evaluating Large Language Models on Process Mining Tasks. In: ICPM Workshops. Lecture Notes in Business Information Processing, vol. 533, pp. 610–623. Springer (2024)
5. Berti, A., Qafari, M.S.: Leveraging Large Language Models (LLMs) for Process Mining (Technical Report). *CoRR* **abs/2307.12701** (2023)
6. Cremerius, J., Pufahl, L., Klessascheck, F., Weske, M.: Event log generation in MIMIC-IV research paper. In: ICPM Workshops. Lecture Notes in Business Information Processing, vol. 468, pp. 302–314. Springer (2022)
7. Diba, K., Batoulis, K., Weidlich, M., Weske, M.: Extraction, correlation, and abstraction of event data for process mining. *WIREs Data Mining Knowl. Discov.* **10**(3) (2020)
8. Gomez-Noe, A., et al.: Extraction and Identification of Medications from EHR Notes with LLMs: A Tool and a Case Study. In: ICPM Workshops (PODS4H). Springer (2025)

9. Heidemeyer, H., Auhagen, L., Majeed, R.W., Pegoraro, M., Bienzeisler, J., Peeva, V., Beyel, H., Röhrig, R., van der Aalst, W.M.P., Puladi, B.: A Pipeline for the Usage of the Core Data Set of the Medical Informatics Initiative for Process Mining - A Technical Case Report. In: German Medical Data Sciences 2024. Studies in Health Technology and Informatics, vol. 317, pp. 30–37. IOS Press (2024)
10. Helm, E., Krauss, O., Lin, A.M., Pointner, A., Schuler, A., Küng, J.: Process Mining on FHIR - An Open Standards-Based Process Analytics Approach for Healthcare. In: ICPM Workshops. Lecture Notes in Business Information Processing, vol. 406, pp. 343–355. Springer (2020)
11. Hripcsak, G., Ryan, P.B. (eds.): The Book of OHDSI. OHDSI (2019)
12. Illueca-Fernandez, E., Chen, K., Seoane, F., Abtahi, F.: HealthProcessAI: a technical framework and proof-of-concept for LLM-enhanced healthcare process mining. *Frontiers Artif. Intell.* **9**, 1716819 (2026)
13. Johnson, A.E.W., Bulgarelli, L., Shen, L., Gayles, A., Shammout, A., Horng, S., Pollard, T.J., Hao, S., Moody, B., Szolovits, P., Celi, L.A., Mark, R.G.: MIMIC-IV, a freely accessible electronic health record dataset. *Scientific Data* **10**, 1 (2023)
14. Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., Zhou, X., Ma, C., Li, G., Chang, K.C., Huang, F., Cheng, R., Li, Y.: Can LLM Already Serve as a Database Interface? A BIG Bench for Large-Scale Database Grounded Text-to-SQLs. In: *NeurIPS* (2024)
15. Munoz-Gama, J., et al.: Process mining for healthcare: Characteristics and challenges. *J. Biomed. Informatics* **127**, 103994 (2022)
16. Naeem, M.R., Naeem, H., Aamir, M., Ali, W., Abro, W.A.: A multi-level process mining framework for correlating and clustering of biomedical activities using event logs. *International Journal of Advanced Computer Science and Applications* **8**(3), 393–401 (2017)
17. Overhage, J.M., Ryan, P.B., Reich, C.G., Hartzema, A.G., Stang, P.E.: Validation of a common data model for active safety surveillance research. *Journal of the American Medical Informatics Association* **19**(1), 54–60 (2012)
18. Park, G., Lee, Y., Cho, M.: Enhancing healthcare process analysis through object-centric process mining: Transforming OMOP CDM into OCEL. *J. Biomed. Informatics* **156**, 104682 (2024)
19. Park, K., Cho, M., Song, M., Yoo, S., Baek, H., Kim, S., Kim, K.: Exploring the potential of OMOP common data model for process mining in healthcare. *Plos one* **18**(1), e0279641 (2023)
20. Rebuge, Á., Ferreira, D.R.: Business process analysis in healthcare environments: A methodology based on process mining. *Inf. Syst.* **37**(2), 99–116 (2012)
21. Stein Dani, V., Dees, M., Leopold, H., Busch, K., Beerepoot, I., van der Werf, J.M.E.M., Reijers, H.A.: Event Log Extraction for Process Mining Using Large Language Models. In: *CoopIS. Lecture Notes in Computer Science*, vol. 15506, pp. 56–72. Springer (2024)
22. Verbeek, H.M.W., Buijs, J.C.A.M., van Dongen, B.F., van der Aalst, W.M.P.: XES, XESame, and ProM 6. In: *CAiSE Forum (Selected Papers). Lecture Notes in Business Information Processing*, vol. 72, pp. 60–75. Springer (2010)
23. Vidgof, M., Bachhofner, S., Mendling, J.: Large Language Models for Business Process Management: Opportunities and Challenges. In: *BPM (Forum). Lecture Notes in Business Information Processing*, vol. 490, pp. 107–123. Springer (2023)
24. Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., Radev, D.: Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In: *EMNLP*. pp. 3911–3921. ACL (2018)